

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	S.Krishna Chaithanya	Reg. No.:	192424287
Section / Batch:	B.Tech-AIDS/2024	Date:	03/08/2026

Code of Conduct

I S.Krishna Chaithanya (192424287) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Krishna Chaithanya

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
 - Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow the \mid a$

$N \rightarrow student \mid teacher \mid book$

$V \rightarrow reads \mid likes$

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
```

```
    pass
```

Test Cases

Input	Expected	Output
-------	----------	--------

the student reads a book	Valid	Parse Tree
--------------------------	-------	------------

a teacher likes the book	Valid	Parse Tree
--------------------------	-------	------------

student reads book	Invalid	Sentence
--------------------	---------	----------

the book likes a teacher	Valid	Parse Tree
--------------------------	-------	------------

reads the student book	Invalid	Sentence
------------------------	---------	----------

Aim

To design and implement a Python program that validates whether an input sentence follows the given Context-Free Grammar (CFG) rules and constructs its parse tree representation. If the sentence does not satisfy the grammar rules, the program displays "**Invalid Sentence**".

Algorithm

1. **Start** the program.
2. Define the grammar components:
 - Determiners: the, a
 - Nouns: student, teacher, book
 - Verbs: reads, likes
3. Accept a sentence from the user.
4. Convert the sentence into lowercase and split it into words.
5. Check whether the sentence contains exactly **5 words**.
 - If not, display "**Invalid Sentence**" and stop.
6. Verify whether the words follow the CFG pattern:
 - **$S \rightarrow NP VP$**
 - **$NP \rightarrow Det N$**
 - **$VP \rightarrow V NP$**
7. If all words satisfy the grammar rules:
 - Display "**Valid Parse Tree**".
 - Print the parse tree.
8. Otherwise, display "**Invalid Sentence**".
9. **Stop** the program.

Code:

```
def parse_cfg(sentence):
    words = sentence.lower().split()
    determiners = ["the", "a"]
    nouns = ["student", "teacher", "book"]
    verbs = ["reads", "likes"]

    if len(words) != 5:
        print("Invalid Sentence")
        return
```

```
det1, noun1, verb, det2, noun2 = words
```

```
if (det1 in determiners and  
    noun1 in nouns and  
    verb in verbs and  
    det2 in determiners and  
    noun2 in nouns):
```

```
    print("\nValid Parse Tree\n")  
    print("        S")  
    print("      /  \")  
    print("    NP   VP")  
    print("  / \  /  \")  
    print("Det N V  NP")  
    print(" | | | /  \")  
    print(f" {det1} {noun1} {verb} Det N")  
    print("      |  |")  
    print(f"      {det2} {noun2}")
```

```
else:
```

```
    print("Invalid Sentence")
```

```
# Main Program
```

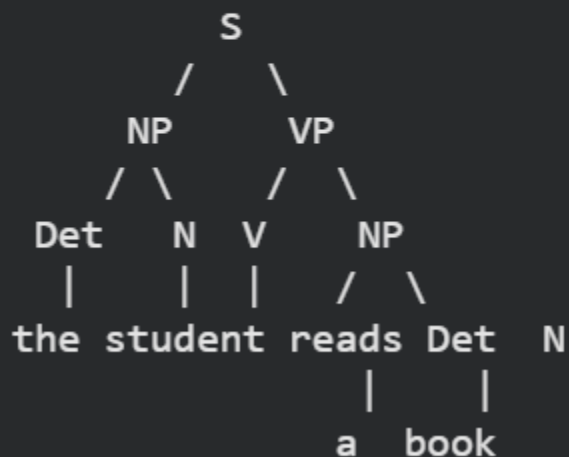
```
sentence = input("Enter a sentence: ")
```

```
parse_cfg(sentence)
```

Output:

```
... Enter a sentence: the student reads a book
```

```
Valid Parse Tree
```



Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he    → singular, third person
she   → singular, third person
it    → singular, third person
they  → plural
```

```
runs  → singular verb
writes → singular verb
```

```
run    → plural verb
write  → plural verb
```

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
---------	------	----------

he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Aim

To design and implement a Python program that verifies subject–verb agreement using feature structures based on Number (Singular/Plural) and Person (First/Second/Third). The program checks whether the given subject and verb agree according to the specified grammar rules and returns True if they match; otherwise, it returns False.

Algorithm

1. Start the program.
2. Define the feature structures for subjects:
 - he → Singular, Third Person
 - she → Singular, Third Person
 - it → Singular, Third Person
 - they → Plural
3. Define the feature structures for verbs:
 - runs, writes → Singular verbs

- run, write → Plural verbs

4. Accept the subject and verb from the user.
5. Check whether the subject and verb exist in the predefined feature structures.
6. Compare the Number feature of the subject with the corresponding verb type.
7. If the features match, return True.
8. Otherwise, return False.
9. Stop the program.

Code:

```
def check_agreement(subject, verb):
    subject_features = {
        "he": ("singular", "third"),
        "she": ("singular", "third"),
        "it": ("singular", "third"),
        "they": ("plural", "third")
    }

    # Verb Feature Structure
    verb_features = {
        "runs": "singular",
        "writes": "singular",
        "run": "plural",
        "write": "plural"
    }

    subject = subject.lower()
    verb = verb.lower()

    if subject not in subject_features or verb not in verb_features:
        return False

    subject_number = subject_features[subject][0]
    verb_number = verb_features[verb]

    return subject_number == verb_number

subject = input("Enter Subject: ")
verb = input("Enter Verb: ")

print(check_agreement(subject, verb))
```

Output:

```
... Enter Subject: he
Enter Verb: runs
True
```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

$S \rightarrow NP \ VP \quad [1.0]$

$NP \rightarrow \text{John} \quad [0.6]$

$NP \rightarrow \text{Mary} \quad [0.4]$

$VP \rightarrow \text{runs} \quad [0.5]$

$VP \rightarrow \text{walks} \quad [0.5]$

Probability Formula

$P(\text{Sentence})$

$=$

$P(S \rightarrow NP \ VP)$

$\times P(NP)$

$\times P(VP)$

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):  
    pass
```

Test Cases

Input Sentence Expected Probability

John runs 0.30

John walks 0.30

Mary runs 0.20

Mary walks 0.20

Peter runs 0.00

Aim

To design and implement a Python program that parses a two-word sentence using a Probabilistic Context-Free Grammar (PCFG) and computes its probability. If the sentence cannot be generated by the given grammar, the program returns 0.00.

Algorithm

1. **Start** the program.
2. Define the PCFG rules:
 - $S \rightarrow NP VP$ with probability **1.0**
 - $NP \rightarrow John$ with probability **0.6**
 - $NP \rightarrow Mary$ with probability **0.4**
 - $VP \rightarrow runs$ with probability **0.5**
 - $VP \rightarrow walks$ with probability **0.5**
3. Accept a two-word sentence from the user.
4. Split the sentence into two words.
5. Check whether the sentence contains exactly **two words**.
 - If not, return **0.00**.
6. Verify that:
 - The first word is a valid **NP**.
 - The second word is a valid **VP**.
7. Compute the sentence probability using:
$$P(\text{Sentence}) = P(S \rightarrow NP VP) \times P(NP) \times P(VP)$$
8. Display the computed probability.
9. If the sentence cannot be generated, return **0.00**.
10. **Stop** the program.

Code:

```
def sentence_probability(sentence):
```

```
    start_prob = 1.0
```

```
    np_rules = {
```

```
        "John": 0.6,
```

```
        "Mary": 0.4
```

```
    }
```



```
vp_rules = {  
    "runs": 0.5,  
    "walks": 0.5  
}  
  
words = sentence.split()  
  
if len(words) != 2:  
    return 0.00  
  
subject, verb = words  
  
if subject in np_rules and verb in vp_rules:  
    probability = start_prob * np_rules[subject] * vp_rules[verb]  
    return probability  
  
else:  
    return 0.00  
  
sentence = input("Enter a two-word sentence: ")  
prob = sentence_probability(sentence)  
print("Sentence Probability =", format(prob, ".2f"))
```

Output:

```
... Enter a two-word sentence: John runs  
Sentence Probability = 0.30
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____